

Guia de Defesa do Trabalho TR1

Simulador das Camadas Física e de Enlace

Gustavo Henrique Andrade Cavalcanti

Fernando Augusto Hortencio

Teleinformática e Redes 1 - Universidade de Brasília - 2026

Resumo

Este guia consolida a explicação técnica do Simulador TR1 para a apresentação oral. O sistema transmite uma mensagem de texto entre uma thread transmissora e uma thread receptora, passando pela camada de aplicação, pela camada de enlace, pela camada física e por um meio de comunicação ruidoso. A defesa deve deixar claro que o projeto não é apenas uma interface gráfica: ele implementa, manualmente, os algoritmos centrais de enquadramento, detecção de erros, correção de erros e modulação. O professor pode perguntar tanto sobre o código quanto sobre a teoria, então o objetivo deste documento é conectar cada conceito ao arquivo e à função que o implementam.

Palavras-chave: camada física; camada de enlace; enquadramento; CRC-32; Hamming; QPSK; 16-QAM; AWGN.

1 Objetivo e visão geral

O trabalho simula, de ponta a ponta, o caminho de uma mensagem em uma rede: o texto é convertido em bits, os bits são organizados em quadros, os quadros recebem redundância para lidar com erros, e a camada física transforma esses bits em amostras de sinal elétrico em Volts. Depois disso, o meio de comunicação soma ruído gaussiano ao sinal. No receptor, o processo é invertido: o sinal é demodulado, os quadros são recuperados, erros são corrigidos ou detectados e os bits voltam a formar texto.

A ideia central de defesa é: **bits não trafegam no meio como objetos abstratos**. O que passa pelo canal é uma forma de onda, sujeita a ruído, potência, amplitude, frequência e fase. O projeto torna isso visível ao mostrar bits, quadros, sinais transmitidos, sinais recebidos, relatório de erros e texto final.

O fluxo conceitual é:

texto → bits → quadros → sinal → canal ruidoso → sinal recebido → bits → texto.

A separação entre transmissor e receptor é feita em `simulador.py` com duas threads e filas `queue.Queue`. O canal fica entre as duas: ele recebe o sinal limpo da thread TX, aplica ruído em `meio_comunicacao.py` e entrega o sinal ruidoso à thread RX. Isso atende ao espírito do enunciado: o TX e o RX são entidades separadas, e o meio é o único ponto de contato.

2 Camada de enlace: quadros, detecção e correção

2.1 Enquadramento

A camada física entrega ao receptor um fluxo contínuo de bits. A camada de enlace precisa saber onde cada quadro começa e termina. Por isso o projeto implementa três técnicas de enquadramento em `camada_enlace.py`.

Técnica	Como funciona e o que defender
Contagem de caracteres	O quadro começa com um byte que informa quantos bytes de payload vêm em seguida. É simples e tem baixo overhead, mas se o campo de contagem for corrompido, o receptor perde o alinhamento e pode interpretar os quadros seguintes de forma errada. No código: <code>enquadrar_contagem</code> e <code>desenquadrar_contagem</code> .
FLAGS com inserção de bytes	Usa FLAG 0x7E para delimitar quadros e ESC 0x7D para proteger bytes especiais que aparecem no payload. Se o dado contém FLAG ou ESC, o transmissor insere ESC antes dele. No receptor, uma pequena máquina de estados distingue dado literal de delimitador. No código: <code>enquadrar_bytes</code> e <code>desenquadrar_bytes</code> .
FLAGS com inserção de bits	Usa a FLAG 01111110. Para impedir que o payload imite essa sequência, o transmissor insere um bit 0 depois de cinco bits 1 consecutivos. O receptor remove esse 0 extra. No código: <code>enquadrar_bits</code> e <code>desenquadrar_bits</code> .

Perguntas prováveis: por que inserir 0 depois de cinco bits 1? Porque a FLAG tem seis bits 1 consecutivos; cortando a sequência no quinto 1, a FLAG nunca aparece dentro dos dados. Por que escapar também o ESC no byte stuffing? Porque, se ESC pudesse aparecer como dado sem ser escapado, o receptor não saberia se ele é controle ou informação.

2.2 Detecção de erros

Detecção de erros significa anexar ao quadro uma redundância calculada sobre os dados. No receptor, a redundância é recalculada ou validada. Se não bater, houve erro. O projeto implementa três EDCs (*Error Detecting Codes*).

EDC	Custo	Explicação de defesa
Paridade par	1 byte no projeto	A paridade faz o número total de bits 1 ficar par. Detecta qualquer quantidade ímpar de erros, mas falha quando uma quantidade par de bits é invertida. Foi armazenada como 1 byte, e não 1 bit solto, para manter alinhamento com os enquadramentos por bytes.
Checksum	16 bits	Usa soma em complemento de 1 sobre palavras de 16 bits, com <i>end-around carry</i> . No receptor, a soma do payload com o checksum deve fechar em 0xFFFF. É mais forte que paridade, mas ainda pode falhar quando erros se compensam aritmeticamente.
CRC-32	32 bits	Interpreta os bits como polinômios e usa divisão em GF(2), onde soma e subtração são XOR. O projeto implementa o CRC-32 IEEE 802 bit a bit, sem <code>zlib</code> . Usa o polinômio refletido 0xEDB88320, inicialização 0xFFFFFFFF e XOR final. O vetor "123456789" gera 0xCBF43926.

A ordem no transmissor é importante: primeiro o projeto adiciona o EDC e depois aplica Hamming. Assim, o Hamming protege também os bits do verificador. No receptor, a ordem é invertida: primeiro tenta corrigir com Hamming, depois verifica o EDC.

2.3 Correção com Hamming(8,4)

O projeto usa Hamming(8,4) estendido, também chamado SECDED: *Single Error Correction, Double Error Detection*. Cada bloco de 4 bits de dados vira 8 bits:

$$p_1 \ p_2 \ d_1 \ p_4 \ d_2 \ d_3 \ d_4 \ p_0.$$

Os bits p_1 , p_2 e p_4 formam as paridades clássicas de Hamming. A combinação das paridades quebradas forma a síndrome, que aponta a posição do erro simples. O bit p_0 é a paridade geral do bloco e permite detectar erro duplo.

Síndrome	Paridade geral	Diagnóstico
0	correta	Não há erro detectado.
$\neq 0$	quebrada	Há erro simples em uma das posições 1 a 7. O receptor inverte o bit apontado pela síndrome.
0	quebrada	O erro está apenas no bit p_0 . Os dados estão íntegros.
$\neq 0$	correta	Há erro duplo. O código detecta, mas não corrige.

Por que Hamming(8,4), e não Hamming(7,4)? Porque o bit extra p_0 permite detectar erro duplo, além de manter alinhamento em bytes. O custo é alto: 4 bits viram 8 bits, ou seja, a redundância dobra o tamanho do payload protegido.

3 Camada física: de bits para sinais

3.1 Codificação em banda-base

Na banda-base, os bits são representados diretamente por níveis de tensão. O projeto implementa três códigos de linha em `camada_fisica.py`.

- **NRZ-Polar:** bit 1 vira $+V$ e bit 0 vira $-V$. A demodulação usa a média das amostras de cada bit: média positiva indica 1, média negativa ou nula indica 0.
- **Manchester:** sempre há transição no meio do bit, o que ajuda no sincronismo. A convenção adotada é a de Tanenbaum/G.E. Thomas: 0 é transição baixo-para-alto e 1 é alto-para-baixo. A demodulação compara a média da primeira metade do bit com a da segunda metade.
- **Bipolar AMI:** bit 0 vira 0 V; cada bit 1 alterna entre $+V$ e $-V$. Isso reduz a componente DC do sinal. A demodulação usa $|média| > V/2$ para decidir se houve um 1.

3.2 Modulação por portadora

Na modulação por portadora, os bits controlam amplitude, frequência ou fase de uma senoide. A ideia que unifica QPSK e 16-QAM no projeto é a representação I/Q:

$$s(t) = I \cos(2\pi ft) - Q \sin(2\pi ft).$$

Escolher um par (I, Q) é escolher um ponto da constelação. A função `onda(i, q, ciclos)` gera o símbolo correspondente. No receptor, **correlacionar** multiplica o símbolo recebido por cosseno e seno de referência e soma os produtos. Como seno e cosseno são ortogonais ao longo de ciclos inteiros, a correlação recupera as componentes I e Q.

- **ASK, 1 bit/símbolo:** varia a amplitude. O bit 1 transmite portadora com amplitude V ; o bit 0 transmite 0 V. É simples, mas sensível a ruído de amplitude.
- **FSK, 1 bit/símbolo:** varia a frequência. O projeto usa 2 ciclos/símbolo para 0 e 4 ciclos/símbolo para 1. O receptor compara a energia nas duas frequências.
- **QPSK, 2 bits/símbolo:** usa 4 fases e mapeamento Gray. Cada símbolo representa um par de bits. Como vizinhos diferem em 1 bit, um erro para o vizinho tende a causar apenas 1 bit errado.
- **16-QAM, 4 bits/símbolo:** usa uma grade 4x4, variando amplitude e fase. Carrega mais bits por símbolo, mas os pontos ficam mais próximos; por isso exige melhor SNR que QPSK.

O trade-off de modulação é essencial: mais bits por símbolo aumentam eficiência espectral, mas reduzem a distância entre pontos da constelação. Ruído desloca o ponto recebido; a decisão por mínima distância funciona enquanto o deslocamento não ultrapassa a fronteira do símbolo vizinho.

3.3 Meio e ruído

O meio de comunicação está em `meio_comunicacao.py`. Ele soma `random.gauss(media, sigma)` a cada amostra do sinal. Isso modela AWGN: ruído aditivo, branco e gaussiano. A média x pode representar deslocamento DC; o desvio padrão σ controla a intensidade do ruído. A potência média é calculada como $P = (1/N) \sum v^2$, assumindo carga de 1Ω . Essa potência permite comparar sinal e ruído e explicar SNR.

4 Como o código se organiza

Arquivo	O que explicar se o professor abrir o código
<code>camada_aplicacao.py</code>	<code>texto_para_bits</code> percorre bytes UTF-8 e emite bits do mais significativo ao menos significativo. <code>bits_para_texto</code> reagrupa bits em bytes e usa <code>errors="replace"</code> para não travar quando o ruído corrompe uma sequência UTF-8.
<code>camada_enlace.py</code>	Contém conversões bits/bytes, três enquadramentos, três EDCs, Hamming(8,4) e as fachadas <code>transmitir</code> e <code>receber</code> . No TX: divide em blocos, adiciona EDC, aplica Hamming e enquadra. No RX: desenquadra, corrige, verifica EDC e devolve bits da aplicação.
<code>camada_fisica.py</code>	Define $V = 1$, 100 amostras por bit/símbolo, códigos de linha, portadoras, constelações e demoduladores. As funções <code>onda</code> , <code>correlacionar</code> , <code>bits_do_ponto_mais_proximo</code> e <code>decidir_nivel_gray</code> são o núcleo das modulações por portadora.
<code>meio_comunicacao.py</code>	Soma ruído gaussiano ao sinal e calcula potência média. É onde o canal deixa de ser ideal.
<code>simulador.py</code>	Cria threads TX/RX, usa filas para representar o meio, aplica ruído entre as threads e armazena resultados intermediários para a interface.
<code>interface_gui.py</code>	Implementa a GUI em GTK 3, com fallback em Tk. Permite escolher enquadramento, EDC, Hamming, modulação, ruído e tamanho de quadro, além de mostrar bits, sinais, texto recuperado e relatório por quadro.
<code>testes.py</code>	Valida aplicação, enquadramentos, EDCs, CRC conhecido, Hamming com erro simples/duplo, modulações com ruído e simulações completas.

5 Como demonstrar em sala

1. Comece com a tese. “O simulador mostra a transmissão completa: texto vira bits, bits viram quadros, quadros viram sinal, o canal injeta ruído e o receptor tenta reconstruir a mensagem.”

2. Rode os testes. No diretório do projeto:

```
python3 testes.py
```

Explique que os testes cobrem ida e volta UTF-8, casos críticos de stuffing, paridade/checksum/CRC, CRC-32 de referência 0xCB43926, Hamming corrigindo 1 erro e detectando erro duplo, todas as modulações e 15 simulações completas.

3. Abra a GUI.

```
python3 simulador.py
```

Use uma configuração segura para demonstração: enquadramento por bits, detecção CRC, correção Hamming, NRZ-Polar, QPSK, média 0 e $\sigma = 0,10$. Mostre a aba TX, a aba RX, os bits de enlace, o sinal transmitido, o sinal recebido e o relatório por quadro.

4. Aumente o ruído aos poucos. Primeiro o Hamming corrige alguns erros. Depois aparecem erros duplos ou falhas de EDC. Por fim, o texto sai com diferenças. Essa é a melhor demonstração visual do limite entre detecção, correção e capacidade do canal.

5. Mostre três pontos do código. Em `camada_enlace.py`, mostre `transmitir` e `receber`; em `camada_fisica.py`, mostre `onda` e `correlacionar`; em `simulador.py`, mostre `executar_simulacao`.

6 Perguntas prováveis e respostas curtas

- 1. O que o trabalho simula?** Uma transmissão completa entre TX e RX, passando por aplicação, enlace, física e meio ruidoso.
- 2. Por que usar threads?** Para separar transmissor e receptor. As filas representam o meio entre eles.
- 3. Onde o ruído entra?** Depois do TX e antes do RX, somado a cada amostra em `meio_comunicacao.py`.
- 4. Qual a ordem da camada de enlace no TX?** Divide em blocos, adiciona EDC, aplica Hamming e enquadra.
- 5. E no RX?** Desenquadra, corrige com Hamming, verifica EDC e devolve os bits da aplicação.
- 6. Por que Hamming depois do EDC?** Para proteger também os bits do verificador.
- 7. Por que Hamming(8,4)?** Porque corrige erro simples, detecta erro duplo e mantém alinhamento em bytes.
- 8. O que o CRC faz?** Divide a mensagem como polinômio em GF(2); resto diferente indica erro.
- 9. O que é GF(2)?** Aritmética módulo 2, na qual soma e subtração são XOR.
- 10. Por que paridade é fraca?** Porque erros em quantidade par podem passar despercebidos.
- 11. Por que cinco 1s no bit stuffing?** Porque a FLAG tem seis 1s; inserir 0 após cinco impede a imitação da FLAG.
- 12. O que é I/Q?** São componentes ortogonais que representam amplitude e fase de um símbolo.
- 13. Como o RX recupera I/Q?** Por correlação com cosseno e seno de referência.
- 14. Por que Gray em QPSK e 16-QAM?** Para que símbolos vizinhos difiram em apenas 1 bit.
- 15. Qual modulação é mais sensível a ruído?** ASK é muito sensível a amplitude; 16-QAM também exige SNR maior por ter pontos mais próximos.
- 16. Por que 16-QAM carrega 4 bits por símbolo?** Porque possui 16 pontos e $\log_2(16) = 4$.
- 17. O que é SNR no projeto?** Relação entre potência média do sinal e potência média do ruído.
- 18. O que provar se perguntarem sobre corretude?** Rodar `python3 testes.py` e citar os testes de CRC, Hamming, stuffing e simulação completa.

Fontes locais usadas: código e estudos do repositório `TrabalhoTR1`; materiais de camada física e enlace do repositório de Teleinformática e Redes; relatório LaTeX original do grupo; suíte `testes.py`.